# 2042: Interceptor

eslem yildirim

March 2026

**Gameplay Video Link:** `https://youtube.com/shorts/YNHnx7A-K-s?si=LSRKXN6wV-1YFUwi`

## 1 Introduction

*2042: Interceptor* is a top-down, 2D arcade shooter game where the player controls a fighter ship (the Interceptor) to defend against incoming enemy aircraft. The game goes through three different graphical states: A configuration start screen, an active gameplay arena, and a victory or game-over screen.

The game was developed purely in Java utilizing the `StdDraw` library for all graphical rendering, double buffering, and keyboard input processing. To optimize state tracking and memory layout, traditional object-oriented programming (OOP) structures were bypassed in favor of a data-oriented approach. The game's state and entity tracking are managed using primitive data types and parallel `ArrayList` structures (for example, `enemyX` and `enemyY` for enemy coordinates, and `bulletX` and `bulletY` for projectiles).

This parallel array approach ensures that associated data for multiple entities on screen is updated synchronously within the main game loop.

## 2 Game Dynamics

### 2.1 UI and Configuration State

Before the main game loop begins, the player is presented with a configuration screen. This stage allows the user to adjust the game frames per second (FPS) using the A and D keys and the entity movement speed using the Q and E keys. This setup screen gives the player control over the difficulty before pressing ENTER to initialize the main game loop.




Figure 1: The initial start screen displaying the configuration options for FPS and Speed.

## 2.2 Player Controller

Once the game starts, the player controls the Interceptor using the directional arrow keys. Boundary limits are enforced by coordinate checks, preventing the player's ship from leaving the canvas. Table 1 outlines all active keyboard inputs.

Table 1: Player Controls and Game Actions

| **Action** | **Key Binding** |
|---|---|
| Move Interceptor | Up, Down, Left, Right Arrows |
| Shoot Projectile | Spacebar |
| Pause / Unpause Game | P |

Enemy ships spawn in a predetermined formation and move horizontally as a group. During each iteration of the main loop, the program checks whether any enemy has reached the canvas boundary. If a boundary is detected, a boolean flag reverses the global `enemySpeed` variable, causing the entire formation to change direction simultaneously.

## 2.3 Combat System and Collision

When the player presses the Spacebar, the current $x$ and $y$ coordinates of the Interceptor are appended to the bullet arrays, and the bullets travel upward each frame.

Enemies fire autonomously based on a probability model. The firing probability dynamically increases as the number of enemies decreases: when more than three enemies remain, the firing probability is 4%, while when three or fewer enemies remain, the probability increases to 10% to increase game difficulty.

Collisions are calculated using a bounding-box approach around entity center points. By comparing the absolute differences between the coordinates of two entities, the system determines whether their boxes overlap, as defined in Equation 1.

$$|X_1 - X_2| \leq \Delta w \quad \text{and} \quad |Y_1 - Y_2| \leq \Delta h \tag{1}$$

Where $(X_1, Y_1)$ and $(X_2, Y_2)$ represent the center coordinates of the objects, and $\Delta w$ and $\Delta h$ represent the collision thresholds.




Figure 2: Active gameplay showing the Interceptor firing at enemy formations. Destroyed enemies may drop heart items that grant the player additional lives when collected.

## 2.4 Reward System

To help the player survive longer, the game features a reward system. Whenever an enemy ship is destroyed by a player's bullet, there is a 33.3% probability that a heart item will drop from the enemy's position. If the player catches this heart item before it reaches the bottom of the screen, their life counter increases by one. The maximum number of lives the player can hold is limited to five.

## 2.5 The Pause Menu

A pause mechanism allows the player to temporarily stop the game. When the player presses the P key, a boolean variable called `paused` changes its value. When the game is paused, the main game loop enters a nested `while` loop that stops all movements of the game objects. However, the screen continues to display a "Paused" message. This keeps the game state unchanged until the player presses P again to resume the game.

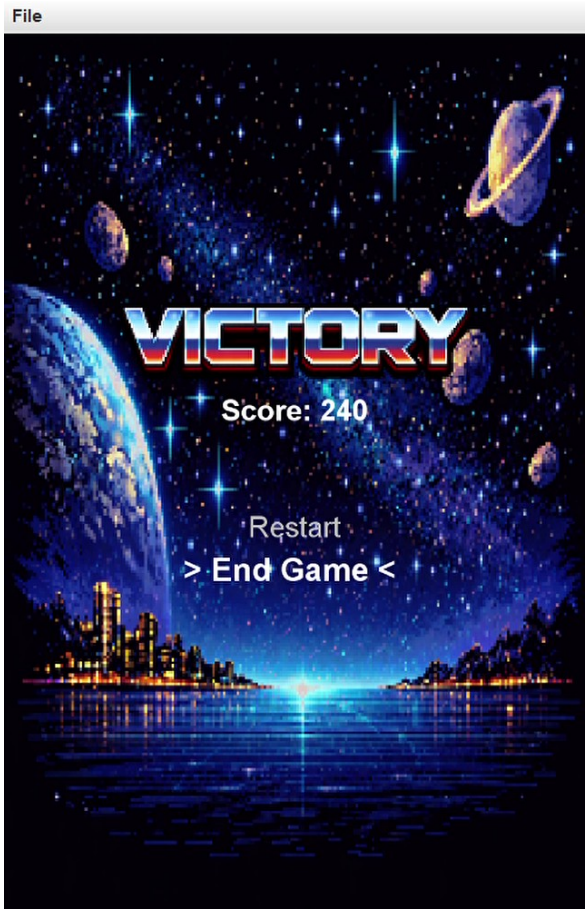



Figure 3: The game in a paused state where all entity movements are halted.

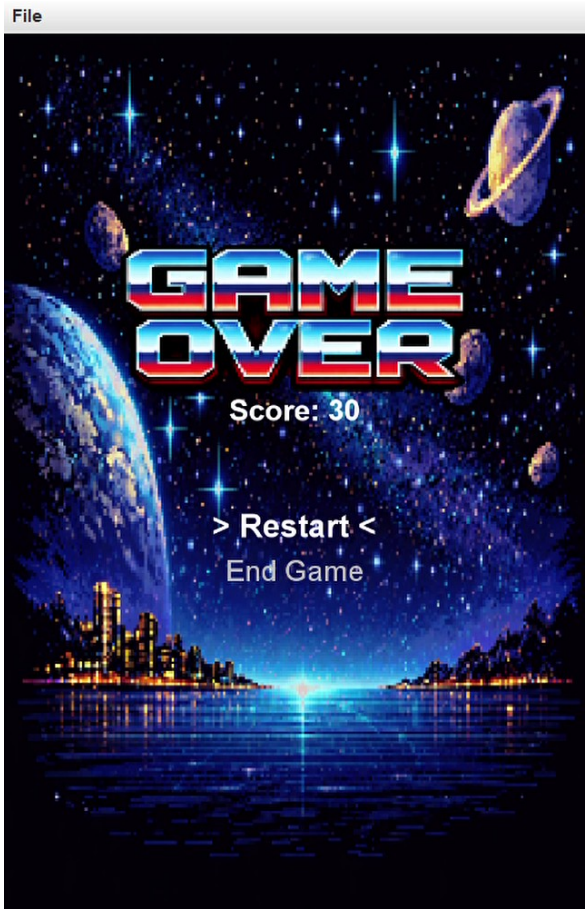
## 2.6 End Game States

The gameplay loop terminates under two conditions: either all enemies are destroyed (resulting in a victory) or the player's life count reaches zero (resulting in a game over). Once one of these conditions is met, the program transitions to an end menu screen.

The end menu displays the final score and provides options for restarting the game or exiting the application. The player can navigate between the menu options using the Up and Down arrow keys. If the player selects Restart, the game starts again. If Exit is selected, the game terminates and the application closes.




(a) Victory screen displayed when all enemies are destroyed.




(b) Game over screen displayed when the player loses all lives.

Figure 4: End game states showing the final score and restart/exit options.